

クラス替えを数学で解く

Claude Code による環境テスト

このサイトは **Claude Code (Anthropic)** を使った WSL2 開発環境のテストとして自動生成されたサンプルコンテンツです。MkDocs・GitHub Pages・TeX (LuaLaTeX) の動作確認を目的としており、実際の授業資料ではありません。

この資料について

整数計画法 (Integer Programming) と Google の OR-tools を使って、**公平で最適なクラス替え**を自動で行う方法を学びます。

 PDF ダウンロード

PDF	説明	ダウンロード
MkDocs 版	このサイトをそのまま PDF 化（WeasyPrint 生成）	 document.pdf
TeX 版	LuaLaTeX で組版した印刷用プリント	 handout-tex.pdf

なぜ「最適化」が必要か

毎年恒例のクラス替え。先生たちは次のようなことを考えながら手作業で組み合わせを考えています。

- 各クラスの**人数をそろえる**
- 男女の**比率をバランス**よくする
- 成績の分布が**偏らない**ようにする
- 仲の悪い生徒が**同じクラスにならない**ようにする

全校生徒 200 人を 5 クラスに分ける組み合わせの数は...

$$\binom{200}{40} \binom{160}{40} \binom{120}{40} \binom{80}{40} \approx 10^{130}$$

これは宇宙の原子の数（約 10^{80} ）よりはるかに大きい数です。 **全探索は不可能**です。だから「数理最適化」の出番です。

このサイトの構成



ページ	内容
問題の定式化	変数・制約・目的関数を数式で書く
Pythonコード	OR-tools CP-SAT ソルバーで解く
結果と考察	解の見方・現実への応用

キーワード

- **整数計画法 (Integer Programming, IP):** 変数が整数（0 or 1）に限定された最適化
- **CP-SAT ソルバー:** Google OR-tools に含まれる高速な制約プログラミングソルバー
- **0-1 変数:** $x \in \{0, 1\}$ — 「する/しない」を表す変数

問題の定式化

数理最適化では、問題を「**変数**」「**制約条件**」「**目的関数**」の3つで表現します。

問題の設定

記号	意味	値
	全生徒数	200 人
	クラス数	5 クラス
	各クラスの最小人数	38 人
	各クラスの最大人数	42 人

決定変数

意味

ならば「生徒 はクラス に所属する」
ならば「所属しない」

変数の総数は 個。

制約条件

① 各生徒は必ず1クラスに所属

「1クラスに重複して所属」や「どこにも所属しない」を禁止します。

② クラスの人数バランス

③ 男女比のバランス

男子生徒の集合を M 、女子生徒の集合を F とします（ $M \cap F = \emptyset$ ）。

④ 仲良しグループの分散（任意）

特定の生徒ペア (i, j) を同じクラスにしない場合：

目的関数

今回は**実行可能解を1つ見つける**ことを目標とします（最小化・最大化の目的関数なし）。

より進んだ設定では、たとえば次のような目的関数を追加できます：

ここで は「同じクラスを避けたいペアの集合」です。ただし、これは**非線形**になるため、補助変数を使って線形化する必要があります。

定式化のまとめ



実行可能性

制約が矛盾していなければ、CP-SAT ソルバーは非常に高速に解を見つけます。1000 変数・1000 制約程度なら通常 **1秒以内** に解けます。

Python コード — OR-tools CP-SAT

インストール

```
pip install ortools
```



完全なコード

```

"""
クラス替え最適化 - OR-tools CP-SAT ソルバー
生徒200人を5クラスに割り当てる整数計画問題
"""

from ortools.sat.python import cp_model
import random

# —— パラメータ

N_STUDENTS = 200      # 全生徒数
N_CLASSES   = 5        # クラス数
CLASS_MIN   = 38       # 各クラスの最小人数
CLASS_MAX   = 42       # 各クラスの最大人数
GENDER_MIN  = 18       # 各クラスの男女それぞれの最小人数
GENDER_MAX  = 22       # 各クラスの男女それぞれの最大人数

# —— ダミーデータ（実際は外部ファイルから読み込む） ——
random.seed(42)
# 生徒の性別: 0=男, 1=女 (各100人)
genders = [0] * 100 + [1] * 100
random.shuffle(genders)

# 「同じクラスを避けたいペア」(例: 3組)
avoid_pairs = [(0, 1), (50, 51), (100, 150)]

# —— モデルの構築

model = cp_model.CpModel()

# 決定変数: x[i][j] = 1 なら生徒i はクラスj に所属
x = [[model.NewBoolVar(f'x_{i}_{j}')
      for j in range(N_CLASSES)]
      for i in range(N_STUDENTS)]

# —— 制約条件

# ① 各生徒は必ず1クラスに所属
for i in range(N_STUDENTS):
    model.AddExactlyOne(x[i][j] for j in range(N_CLASSES))

# ② クラスの人数バランス
for j in range(N_CLASSES):
    class_size = sum(x[i][j] for i in range(N_STUDENTS))
    model.Add(class_size >= CLASS_MIN)
    model.Add(class_size <= CLASS_MAX)

# ③ 男女比のバランス
for j in range(N_CLASSES):
    male_count  = sum(x[i][j] for i in range(N_STUDENTS) if genders[i] == 0)
    female_count = sum(x[i][j] for i in range(N_STUDENTS) if genders[i] == 1)
    model.Add(male_count >= GENDER_MIN)

```

```
model.Add(male_count <= GENDER_MAX)
model.Add(female_count >= GENDER_MIN)
model.Add(female_count <= GENDER_MAX)

# ④ 避けたいペアを別クラスに
for (a, b) in avoid_pairs:
    for j in range(N_CLASSES):
        model.Add(x[a][j] + x[b][j] <= 1)

# —— ソルバーの実行

solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 30.0 # 最大30秒

status = solver.Solve(model)

# —— 結果の出力

if status in (cp_model.FEASIBLE, cp_model.OPTIMAL):
    print("✅ 解が見つかりました！\n")
    for j in range(N_CLASSES):
        members = [i for i in range(N_STUDENTS) if solver.Value(x[i][j]) == 1]
        males = [i for i in members if genders[i] == 0]
        females = [i for i in members if genders[i] == 1]
        print(f"クラス {j+1}: 合計{len(members)}人 "
              f"(男{len(males)}人 / 女{len(females)}人)")
else:
    print("❌ 解が見つかりませんでした（制約が矛盾している可能性があります）")
```


実行結果の例

✓ 解が見つかりました！



クラス 1: 合計40人 (男20人 / 女20人)
クラス 2: 合計40人 (男20人 / 女20人)
クラス 3: 合計40人 (男20人 / 女20人)
クラス 4: 合計40人 (男20人 / 女20人)
クラス 5: 合計40人 (男20人 / 女20人)

コードのポイント解説

NewBoolVar — 0-1 変数の作成

```
x[i][j] = model.NewBoolVar(f'x_{i}_{j}')
```



CP-SAT の `BoolVar` は の値を取ります。これが定式化の に対応します。

AddExactlyOne — ちょうど1つが真

```
model.AddExactlyOne(x[i][j] for j in range(N_CLASSES))
```



を表す便利なメソッドです。

Add — 不等式制約

```
model.Add(class_size >= CLASS_MIN)  
model.Add(class_size <= CLASS_MAX)
```



CP-SAT では線形式を `Add` で制約として追加します。

非線形制約について

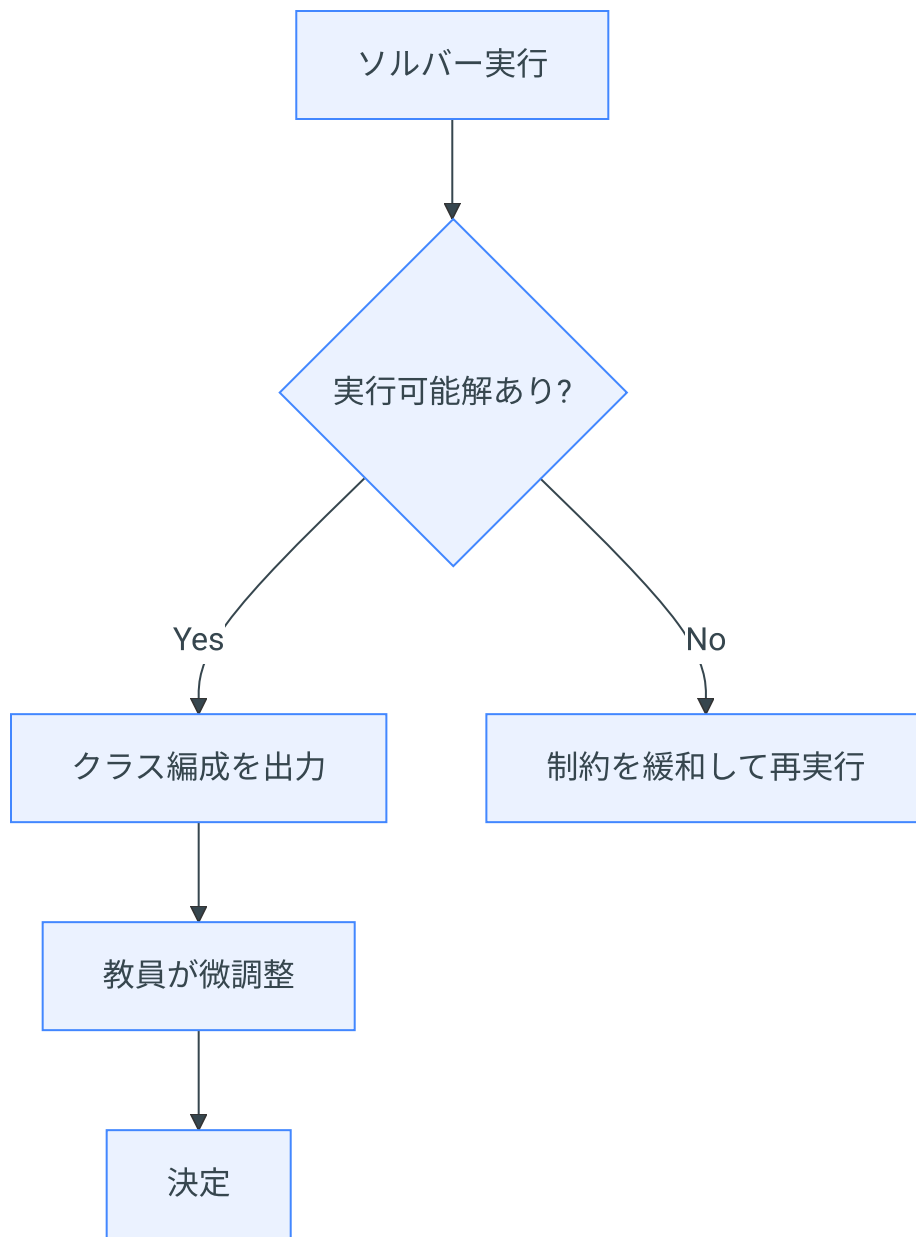
$x[a][j] * x[b][j]$ のような積は直接書けません。補助変数 $z = x[a][j] \text{ AND } x[b][j]$ を使って線形化します。今回の「ペアを分ける」制約は和で表現できるため問題ありません。

結果と考察

計算量の比較

方法	時間	備考
全探索	事実上不可能	組み合わせ数 $\approx 10^{130}$
手作業	数時間～数日	ベテラン教員でも困難
CP-SAT ソルバー	1秒以内	制約が複雑でも対応可

結果の見方



ソルバーが「解なし」と返した場合、制約が厳しすぎる可能性があります。たとえば「避けたいペアが多すぎる」など。制約を段階的に緩めて試します。

発展的な話題

より複雑な目的関数

実際の学校現場では、さらに多くの条件が加わります：

- **部活のバランス**: 各部活の主力選手が分散するよう
- **学力層の分散**: テストのスコア帯が各クラスで均等に
- **過去のクラスメート**: 昨年同じクラスだった生徒は分けるなど

これらも同様に 0-1 変数と線形制約で表現できます。

多目的最適化

複数の目標（人数バランス・男女バランス・学力バランス）を **同時に最適化** したい場合は、**重み付き目的関数**を使います：

$$\text{minimize } w_1 f_1 + w_2 f_2 + w_3 f_3$$

ここで f_1, f_2, f_3 はそれぞれのバランスの「悪さ」を測る指標、 w_1, w_2, w_3 は優先度の重みです。

数理最適化が使われている身近な例

実際の応用例

-  **航空会社のスケジューリング**: 乗務員の勤務割り当て
-  **病院の人員配置**: 看護師のシフト管理
-  **物流**: トラックの積み込みと配送ルート
-  **通信**: 電波の周波数割り当て

クラス替えは、これらと同じ「**割り当て問題 (Assignment Problem)**」の一種です。

まとめ

- クラス替えは 10^{130} 通りの組み合わせを持つ困難な問題
- 整数計画法で「変数・制約・目的関数」として定式化できる
- OR-tools CP-SAT ソルバーを使えば 1 秒以内に解ける
- 現実の複雑な条件も、線形制約として追加できる

” 数学の力

「最適化」は現代社会の至るところで使われています。高校数学で学ぶ「連立不等式」の考え方がその基礎になっています。